



GitHub Trending Top 10

每日热门开源项目深度解读

2026-08-10

今日榜单

- 1 semantica-agi/semantica** Python **NEW**
Graph-Native Infrastructure for Context and Accountable AI Systems
- 2 msitarzewski/agency-agents** Shell **NEW**
A complete AI agency at your fingertips - From frontend wizards to Reddit commun...
- 3 NanmiCoder/MediaCrawler** Python **NEW**
小红书笔记 | 评论爬虫、抖音视频 | 评论爬虫、快手视频 | 评论爬虫、B 站视频 | 评论爬虫、微...
- 4 addyosmani/agent-skills** JavaScript 6天
Production-grade engineering skills for AI coding agents.
- 5 paperclipai/paperclip** TypeScript **NEW**
The open-source app everyone uses to manage agents at work
- 6 PrimeIntellect-ai/prime-agent** TypeScript 3天
A self-improving RLM agent for coding workflows and long-running autonomous task...
- 7 LadybirdBrowser/ladybird** C++ 2天
Truly independent web browser
- 8 ruvnet/RuView** Rust **NEW**
π RuView turns commodity WiFi signals into real-time spatial intelligence, vital...
- 9 danielmiessler/LifeOS** TypeScript **NEW**
A General Hill-climbing AI harness that helps you move from Current State to l...
- 10 firecrawl/firecrawl** TypeScript **NEW**
The context API to search, scrape, and interact with the web at scale.

#1

Python

NEW

semantica

Graph-Native Infrastructure for Context and Accountable AI Systems

Stars **3,619** Forks **405** Today **+967**

<https://github.com/semantica-agi/semantica>

项目简介：Semantica 是一个开源的“图原生”AI 基础设施层，定位为“AI 智能体的 Palantir”。它解决的核心问题是：当前 AI 智能体在做决策时缺乏可审计的上下文和决策轨迹，导致在金融、医疗等强监管场景下无法回答“为什么这么做”。Semantica 通过构建企业数据的关系图谱（Context Graph）和知识图谱，为 LLM 提供可查询、可追溯、可解释的确定性上下文层。

核心功能：一是将企业碎片化数据自动抽取为结构化知识图谱，无需 LLM 参与即可完成图构建；二是提供确定性的推理引擎和图分析能力，支持因果推理与本体管理；三是内置完整的决策溯源机制，从数据输入到决策输出全程留痕；四是支持 RDF 和 LPG 两种图存储标准，兼容 W3C 规范，可与现有数据平台（如 Databricks、Snowflake）无缝集成。

适用场景：主要面向两类团队：一是 AI/ML 平台团队，他们构建的智能体需要处理贷款审批、医疗诊断等高后果决策，要求决策过程可解释、可审计；二是数据平台团队，他们希望将数据仓库中的结构化表转化为带血缘追踪的知识图谱，而不必把数据导出给第三方。典型行业包括金融风控、医疗合规、政府治理等强监管领域。

技术亮点：最值得注意的是它“无 LLM 依赖”的图构建与推理管线——这保证了推理过程完全确定性和可复现，与 LLM 的随机性形成互补。其次，它对 RDF 和 LPG 双标准的支持体现了图存储层面的抽象能力，让上层应用不必绑定特定图数据库。此外，决策溯源（provenance）作为一等公民内建在架构中，而非事后追加，这在同类项目中较为罕见。

上手建议：从 `pip install semantica` 开始，先阅读官方文档（docs.getsemantica.ai）了解核心概念，然后跑通 Knowledge Explorer 的 demo 视频中的示例流程。如果要贡献代码，建议从 GitHub Issues 中标注“good first issue”的任务入手，项目有 CI 和清晰的贡献指南，社区活跃度较高（Discord 可加入讨论）。

#2

Shell

NEW

agency-agents

A complete AI agency at your fingertips - From frontend wizards to Reddit community ninjas, from whimsy injectors to reality checkers. Each agent is a specialized expert with personality, processes, and proven deliverables.

Stars **141,325** Forks **23,051** Today **+858**

<https://github.com/msitarzewski/agency-agents>

项目简介：这是一个将 AI 智能体“人格化”的开源项目，提供了一套完整的 AI 专家团队，从前端开发到社区运营，每个智能体都有独特的个性、工作流程和可交付成果。它解决的问题是让 AI 不再只是通用聊天工具，而是能按专业角色深度参与实际工作的“数字员工”。

核心功能：一是提供大量精心设计的智能体人格文件，覆盖工程、设计、安全、社区运营等多个领域；二是支持一键安装到 Claude Code、Cursor、Codex 等主流 AI 编码工具；三是提供桌面应用（支持 macOS/Linux/Windows），无需命令行即可浏览和安装智能体；四是支持按需选择团队和工具，灵活定制你的“AI 军团”。

适用场景：主要面向使用 AI 辅助开发的程序员和团队，尤其是重度使用 Claude Code、Cursor 等工具的开发人员。适合需要让 AI 承担更专业化、流程化任务的场景，比如前端组件开发、代码安全审查、UI 设计辅助等，也适合想探索 AI 多角色协作的极客用户。

技术亮点：项目用 Shell 脚本实现了多工具兼容层，通过 `convert.sh` 和 `install.sh` 将统一的智能体定义转换为不同 AI 工具所需的格式，设计巧妙。另外，它把智能体设计成“人格+流程+交付物”的三层结构，而非简单的提示词模板，这种结构化设计让 AI 输出更稳定、可预期。

上手建议：最简单的方式是下载桌面应用，浏览智能体列表并一键安装到你的 AI 工具中。喜欢命令行的用户可以直接运行 `./scripts/install.sh` 按向导选择工具和团队。想贡献的话，可以从阅读 `engineering/` 目录下的智能体文件开始，理解其结构后提交新的智能体定义或改进现有流程。

#3

Python

NEW

MediaCrawler

小红书笔记 | 评论爬虫、抖音视频 | 评论爬虫、快手视频 | 评论爬虫、B 站视频 | 评论爬虫、微博帖子 | 评论爬虫、百度贴吧帖子 | 百度贴吧评论回复爬虫 | 知乎问答文章 | 评论爬虫

Stars **60,680** Forks **11,936** Today **+215**

<https://github.com/NanmiCoder/MediaCrawler>

项目简介：MediaCrawler 是一个基于 Python 的多平台自媒体数据采集工具，支持小红书、抖音、快手、B站、微博、贴吧和知乎七大主流平台的内容与评论抓取。它解决了个人开发者或研究者难以高效获取社交媒体公开数据的痛点，提供了统一、低门槛的采集方案。

核心功能：支持关键词搜索、指定帖子 ID 爬取、二级评论抓取、创作者主页数据采集，并内置登录态缓存、IP 代理池和评论词云生成功能。所有平台功能对齐，覆盖从数据采集到可视化的完整链路。

适用场景：适用于社媒舆情分析、竞品监测、市场调研、学术研究等需要批量获取公开数据的场景。个人开发者、数据分析师、产品经理和市场营销人员都能从中受益，尤其适合对爬虫技术不熟悉但需要数据的非专业人士。

技术亮点：巧妙绕过了复杂的 JS 逆向工程，采用 Playwright 浏览器自动化框架保存登录态，并通过 JS 表达式直接获取签名参数，大幅降低了技术门槛。项目还支持 CDP 模式连接用户已有 Chrome 浏览器，复用真实浏览器环境以降低风控风险。

上手建议：建议从 README 的快速开始部分入手，先安装 uv 和 Node.js 依赖，再按照文档配置 Chrome 远程调试。新手可以从关键词搜索功能开始体验，熟悉流程后再尝试高级功能。对于想深入学习架构设计的开发者，可以关注其 Pro 版本的企业级代码设计。

#4

JavaScript

连续 6 天

agent-skills

Production-grade engineering skills for AI coding agents.

Stars **85,438** Forks **9,189** Today **+680**

<https://github.com/addyosmani/agent-skills>

项目简介：

这是一个为 AI 编程代理（如 Claude Code、Cursor、Codex 等）提供“生产级工程技能”的开源项目，本质上是把资深工程师的工作流、质量门槛和最佳实践编码成 AI 可执行的指令集。它解决了 AI 编程中“能写代码但不够规范”的痛点，让代理在开发全流程中遵循统一的高标准。

核心功能：

1. **8 个斜杠命令**：覆盖从 `/spec`（需求定义）到 `/ship`（上线发布）的完整开发生命周期，每个命令自动激活对应技能。
2. **24 个可插拔技能包**：如代码审查、测试驱动开发、API 设计、前端 UI 工程等，按需单独安装。
3. **自动触发机制**：根据当前任务（如设计 API、构建 UI）自动匹配并加载相关技能。
4. **`/build auto` 自治模式**：用户一次性批准计划后，AI 自动逐任务实现、测试并提交，遇错暂停。
5. **跨平台兼容**：通过官方 CLI 一键安装到 70+ 主流 AI 编程工具。

适用场景：

面向所有使用 AI 辅助编程的开发者、技术团队和 AI 工具集成商。尤其适合需要保证代码质量一致性的中大型项目，以及希望将团队工程规范“数字化”并强制执行的组织。个人开发者也能用它提升 AI 生成的代码质量。

技术亮点：

1. **工程经验结构化**：将隐性知识（如“何时写测试”“如何做代码审查”）转化为显式、可复用的技能包。
2. **生命周期全覆盖**：不是零散技巧，而是从需求到上线的完整工作流编排。
3. **轻量级集成**：通过 `npx skills` 单命令安装，或原生插件市场接入，无需改造现有 AI 工具链。
4. **验证前置**：强调“测试即证明”“先测量再优化”等原则，把质量门禁内嵌到 AI 工作流中。

上手建议：

最快速的方式是运行 `npx skills add addyosmani/agent-skills` 一次性安装全部技能，或先

用 `--list` 浏览再按需选择。想深度使用建议从 `/spec` 开始，让 AI 帮你梳理需求再逐步推进。贡献者可从 GitHub 仓库的 `skills/` 目录入手，了解技能包的结构和编写规范，提交新的工程实践技能。

#5

TypeScript

NEW

paperclip

The open-source app everyone uses to manage agents at work

Stars **76,144** Forks **14,175** Today **+167**

<https://github.com/paperclipai/paperclip>

项目简介：

Paperclip 是一个开源的 AI Agent 编排平台，定位是“AI 代理团队的管理系统”——如果把单个 AI 代理（如 OpenClaw、Claude Code）比作员工，Paperclip 就是管理这些员工的“公司”。它通过 Node.js 后端 + React 前端，让用户用任务管理器的交互方式，协调多个 AI 代理共同完成业务目标，并实时监控进度与成本。

核心功能：

1. **任务编排与审批流**：为代理分配目标、设置审批节点，代理完成后由人工验证交付物（如代码 diff）。
2. **组织架构管理**：支持定义“CEO/CTO/工程师”等角色，配置不同代理（OpenClaw、Codex、Cursor 等）的职责与协作关系。
3. **预算与成本控制**：实时追踪每个代理的 API 调用开销，可设定硬性预算上限，超支自动告警。
4. **跨代理协调**：统一管理 20+ 并行终端会话，避免多代理重复工作或目标冲突。
5. **移动端监控**：支持从手机查看任务进度、审批请求和成本报表，实现“遥控”自主业务。

适用场景：

- 开发者或团队同时运行多个 AI 代理（如 20 个 Claude Code 终端）且需要统一管理；
- 想要构建“自主 AI 公司”的创业者，让代理 24/7 协作完成产品开发、营销等任务；
- 需要严格审计 AI 工作过程（如合规要求）或控制 AI 成本的企业用户。

技术亮点：

- **“心跳”集成机制**：任何能接收心跳的代理（包括 Bash 脚本、HTTP 服务）都能接入，极大降低了生态适配门槛；
- **四支柱架构**：围绕“任务、组织、训练、基础设施”四个维度设计，而非简单的任务队列，更贴近真实企业管理模型；

- **TypeScript 全栈**：前后端统一语言，便于社区贡献和类型安全；MIT 协议允许自由商用和二次开发。

上手建议：

- 先阅读 README 的 Quickstart 部分，用 Docker 或本地 Node.js 环境启动一个 demo 实例，体验创建任务、添加代理和审批流；
- 官方文档 docs.paperclip.dev 有详细配置指南，建议重点看“代理接入”章节，了解如何将现有代理（如 Claude Code）注册到平台；
- 想贡献代码可先加入 Discord 社区，从 GitHub Issues 中挑选 good first issue，代码结构清晰（server + UI 分离），TypeScript 开发者上手较快。

#6

TypeScript

连续 3 天

prime-agent

A self-improving RLM agent for coding workflows and long-running autonomous tasks.

Stars **12,442** Forks **1,255** Today **+2,356**

<https://github.com/PrimeIntellect-ai/prime-agent>

项目简介：Prime Agent 是一个开源的编程与研究智能体，专注于长时间运行的自主任务。它通过“递归语言模型”（RLM）和“持续训练框架”（Continual Harness）两大核心抽象，让智能体在持久化环境中不断自我改进，突破单次对话的上下文限制。

核心功能：一是持久化 IPython 环境作为内置工具，所有文件操作、命令执行和上下文管理都通过代码完成；二是内置子代理机制，可并行或后台运行子任务并程序化返回结果；三是支持自我改进，通过 `/refine` 命令基于证据小幅更新辅助状态，且支持回滚；四是技能可执行化，将重复工作流封装为可导入的 Python 包；五是后台守护进程，终端断开后任务继续运行，可随时重新连接。

适用场景：适合需要长时间运行的复杂编码任务、自动化研究流程、多步骤工作流编排，以及需要跨会话保持工作记忆的场景。开发者、数据科学家和 AI 研究者可以用它来构建可复用、可自我优化的自动化工作流水线。

技术亮点：将上下文视为“变量”、子代理视为“函数调用”的 RLM 设计，本质上把智能体交互变成了可编程的 REPL 循环；持续训练框架让辅助状态（记忆、技能、子代理规格）可被智能体自身通过小步更新来优化，而基础系统提示词保持不可变，这种设计兼顾了灵活性与稳定性。

上手建议：从安装最新稳定版开始，在目标项目目录中启动并登录即可体验。建议先在可丢弃的克隆或干净工作区中试用，熟悉 `/refine` 的自我改进机制和 `agents`、`attach` 等会话管理命令。若要贡献，可从阅读 `packages/coding-agent/docs/index.md` 文档入手，了解核心抽象后再参与迭代。

#7

C++

连续 2 天

ladybird

Truly independent web browser

Stars **65,130** Forks **3,112** Today **+190**

<https://github.com/LadybirdBrowser/ladybird>

项目简介：Ladybird 是一个真正独立开发的网页浏览器，目标是构建一个基于 Web 标准、不依赖任何现有浏览器引擎（如 Blink、Gecko）的全新浏览器。它解决了当前浏览器市场由少数巨头主导、引擎技术高度同质化的问题，试图从零开始打造一个开放、透明的浏览器生态。

核心功能：多进程架构，每个标签页拥有独立的渲染进程并与其他系统进程隔离，提升安全性和稳定性；内置完整的 Web 渲染引擎（LibWeb）和 JavaScript 引擎（LibJS），以及对 WebAssembly、TLS、HTTP/1.1 等核心网络与安全协议的原生支持；支持图像解码、音视频播放等多媒体能力，并具备跨平台运行能力（Linux、macOS、Windows WSL2）。

适用场景：当前主要面向浏览器开发者和 Web 标准研究者，用于探索浏览器引擎的实现细节、参与开源贡献或进行安全研究。对于普通用户，由于项目仍处于 pre-alpha 阶段，不适合日常使用，但可作为技术爱好者的“玩具”项目来体验最前沿的浏览器架构。

技术亮点：完全自主的引擎栈是一大亮点，从渲染、JS 执行到网络层全部自研，不依赖 Chromium 或 Firefox 的代码；多进程 + 沙箱设计将图像解码和网络请求放到独立进程，增强了对抗恶意内容的鲁棒性；继承自 SerenityOS 的模块化库（如 LibWeb、LibJS）设计清晰，便于独立测试和复用。

上手建议：如果你是开发者，建议先阅读 Documentation/BuildInstructionsLadybird.md 完成编译，再浏览 Documentation/ 目录下的架构说明。想贡献代码的话，可以加入 Discord 社区，从 CONTRIBUTING.md 中的 issue 政策开始，挑选标注为“good first issue”的任务入手。注意项目处于早期阶段，参与前需对 C++ 和浏览器底层有一定了解。

#8

Rust

NEW

RuView

π RuView turns commodity WiFi signals into real-time spatial intelligence, vital sign monitoring, and presence detection — all without a single pixel of video.

Stars **89,145** Forks **11,872** Today **+156**

<https://github.com/ruvnet/RuView>

项目简介： π RuView 是一个基于 WiFi 信号的非接触式空间感知平台。它利用普通路由器发射的无线电波，通过分析人体运动对信号产生的扰动（CSI 数据），实现穿墙感知人体存在、呼吸与心率监测等功能，全程无需摄像头或穿戴设备。

核心功能：① 人员存在检测与计数，可穿墙追踪进出；② 生命体征监测，无接触测量呼吸率和心率；③ 活动识别，区分行走、静坐、跌倒等动作；④ 环境射频指纹定位，识别房间及物体移动；⑤ 睡眠质量分析，含睡眠分期与呼吸暂停筛查。

适用场景：智慧养老（跌倒检测、久卧提醒）、智能家居（按房间语音播报人员与状态）、安防监控（穿墙存在探测）、医疗健康（无感生命体征监测）。适合家庭用户、养老机构、医院及智能家居开发者使用。

技术亮点：项目将 WiFi 感知提升到“人体姿态估计”级别——可从 CSI 数据重建 17 个身体关键点，实现无摄像头的人体骨架识别。同时采用 Rust 语言构建高性能信号处理管道，并融合 MQTT、Matter、HAP 等多种协议，原生对接主流智能家居生态，具备完善的隐私治理机制（不确定性标注、证据溯源）。

上手建议：新手可从 Home Assistant 集成开始，使用 `--mqtt` 标志即可快速接入现有智能家居环境；开发者建议先阅读 `docs/integrations/home-assistant.md` 了解架构，再通过 `npx @ruvnet/ruview` 工具进行环境检查与模型加载。硬件方面需准备 ESP32 传感器节点，软件层面可参与模型训练或 LoRA 配置文件开发。

#9

TypeScript

NEW

LifeOS

A General Hill-climbing AI harness that helps you move from Current State to Ideal State in both Life and Work.

Stars **17,601** Forks **2,358** Today **+143**

<https://github.com/danielmiessler/LifeOS>

项目简介：LifeOS 是一个通用型 AI 辅助框架（AI Harness），核心理念是帮助用户从“当前状态”迈向“理想状态”。它通过捕获用户的身份、价值观与目标，让 AI 在充分了解用户的前提下，协助完成生活与工作中的各类任务——从写代码到创业再到创意项目，本质上是给 AI 装上一个“了解你的大脑”。

核心功能：1) **意图工程（Intent Engineering）**——将模糊想法转化为可执行的 AI 指令；2) **通用爬山算法（General Hill Climbing）**——系统性地迭代逼近理想状态；3) **TELOS 框架**——目标拆解与执行追踪；4) **上下文系统（Cortex/Synapse/Atlas）**——记忆、知识图谱与信息检索；5) **技能与钩子系统（Skills/Hooks）**——可扩展的能力模块和自动化触发机制。

适用场景：适合重度使用 AI 编码工具的开发者、独立创业者、知识工作者，以及任何希望将 AI 深度融入个人效能系统的人。典型场景包括：用 Claude Code 管理复杂项目、建立个人知识库、自动化日常工作流、系统化推进长期目标等。

技术亮点：采用 TypeScript + Bun 构建，安装方式极具创新——用户只需将安装提示词粘贴给 AI 编码助手，AI 会自动完成全部配置。项目内置了“Pulse”社区系统和“Hermes Sidecar”扩展机制，且架构上强调“只保留独特价值”，避免与现有 AI 工具链重复造轮子，体现了清晰的设计哲学。

上手建议：最简单的方式是打开 Claude Code（或 Cursor 等支持 agent 的编码工具），粘贴 `Read https://ourlifeos.ai/install and install LifeOS for me.` 这行提示词，AI 会引导完成安装。想贡献代码的话，建议先阅读文档了解 TELOS 和意图工程的核心概念，再从 GitHub Issues 中寻找标注 `good first issue` 的任务入手。

#10

TypeScript

NEW

firecrawl

The context API to search, scrape, and interact with the web at scale.

Stars **164,531** Forks **9,259** Today **+815**

<https://github.com/firecrawl/firecrawl>

项目简介：Firecrawl 是一个面向 AI 时代的网页数据抓取 API，能够将任意网站内容转换为干净的 Markdown 或结构化 JSON 数据，解决了大模型应用中“网页数据获取难、清洗难”的核心痛点。项目以 TypeScript 编写，提供开源版本和云服务，目前已有超过 16 万星标，是当下最热门的 Web 数据基础设施之一。

核心功能：其一，网页搜索与抓取，支持将搜索结果直接转化为 LLM 可读格式；其二，智能爬虫，可一键抓取整个网站的所有 URL；其三，站点地图发现，快速获取网站链接结构；其四，批量异步抓取，支持数千 URL 的并发处理；其五，交互式操作，支持点击、滚动、输入等模拟行为后再提取内容。

适用场景：主要面向 AI 应用开发者、数据科学家和 RAG（检索增强生成）系统构建者。典型场景包括：为 LLM 提供实时网页上下文、构建知识库、监控竞品网站变化、自动化数据采集，以及为智能体（Agent）提供网络访问能力。凡是需要“让 AI 读懂网页”的需求，都是其目标场景。

技术亮点：项目宣称能覆盖 96% 的网页（包括 JavaScript 重度渲染的页面），P95 延迟仅 3.4 秒，这背后是自动代理轮换、反爬虫绕过、速率限制处理等复杂工程能力。其输出专为 LLM 优化，能显著减少 token 消耗，同时提供结构化数据提取，技术深度体现在“把脏活累活都封装好了”的工程实现上。

上手建议：最简单的方式是前往 firecrawl.dev 注册获取 API Key，然后通过 Python 或 Node.js SDK 几行代码即可体验。若想本地部署或贡献代码，可直接从 GitHub 仓库克隆，项目文档完善，社区活跃（Discord 可加入）。建议先跑通 Search 和 Scrape 两个核心接口，再逐步探索 Crawl 和 Agent 等高级功能。

数据来源: github.com/trending

报告日期: 2026-08-10

由 DeepSeek AI 提供智能分析 | 自动生成